A3: Epidemic Spreading on Complex Networks

Complex Networks

Universitat Rovira i Virgili (URV)
Master in Artificial Intelligence (MAI)

Andreu Garcies Ramon
andreu.garcies@estudiantat.upc.edu

Maria Guasch Torres
maria.guasch.t@estudiantat.upc.edu

April 2026

1 Introduction

2 The SIS model

The Susceptible-Infected-Susceptible (SIS) model is one of the simplest compartmental models for epidemic spreading. Each node in the network occupies one of two states at any given time:

- **Susceptible (S):** the node is healthy but can be infected.
- **Infected (I):** the node carries the disease and can transmit it to neighbors.

The dynamics are governed by two parameters:

1. $\beta \in [0, 1]$ is the transmission probability: at each discrete time step, an infected node transmits the disease to each susceptible neighbour independently with probability β.
2. $\mu \in [0, 1]$ is the recovery probability: at each time step, an infected node recovers (returns to the susceptible state) with probability μ. Because recovered nodes are susceptible again, the model admits a stationary endemic state in which a non-zero fraction of the population remains infected indefinitely.

The key macroscopic observable is the stationary infection density (the parameter that we will estimate in the experiments):

$$\rho = \frac{1}{N} \sum_{i=1}^{N} \rho_i, \tag{1}$$

where ρ_i is the long-run probability that node i is infected. An important characteristic of the SIS model is the existence of an epidemic threshold β_c. For $\beta < \beta_c$ the disease dies out ($\rho \to 0$), while for $\beta > \beta_c$ it persists at a positive endemic level.

2.1 Monte Carlo simulations

Monte Carlo (MC) simulations implement the SIS dynamics stochastically and average the resulting infection density over many independent realizations. The simulation is run for $T_{\max}$ time steps per realisation. The first T_{trans} steps are discarded as a transient, and ρ is estimated by averaging over the remaining $T_{\max} - T_{\text{trans}}$ steps. The procedure is repeated over N_{rep} independent realizations, each starting from a fresh random infection of a fraction R_0 of nodes, and the final estimate is the mean over all realizations.

Algorithm 1 shows the straightforward implementation. At each time step, the algorithm iterates over every susceptible node and checks, for each of its neighbors, whether that neighbour is infected and whether transmission occurs. The inner loop therefore touches every edge incident to a susceptible node, yielding a per-step cost of $O(N + E)$. With $N_{\text{rep}} \times T_{\max}$ steps across all realizations the total cost is $O(N_{\text{rep}} \cdot T_{\max} \cdot (N + E))$.

The parameters used throughout this work are $N_{\text{rep}} = 100$, $R_0 = 0.2$, $T_{\max} = 1000$, and $T_{\text{trans}} = 900$, giving 100 post-transient steps per realisation.

Algorithm 1 SIS Monte Carlo simulation (naive)

Require: Graph $G = (V, E)$, infection probability β, recovery probability μ, repetitions N_{rep}, initial fraction R_0, steps $T_{\max}$, transient T_{trans}

Ensure: Stationary infection density ρ

```
ρ_avg ← 0
for rep = 1 to N_rep do
    infected ← random sample of ⌊R_0 · N⌋ nodes
    susceptible ← V \ infected
    ρ_rep ← 0
    for t = 1 to T_max do
        if infected = ∅ then break
        end if
        recoveries ← ∅
        for v ∈ infected do
            if random() < μ then    recoveries ← recoveries ∪ {v}
            end if
        end for
        new_infected ← ∅
        for v ∈ susceptible do
            for u ∈ N(v) do
                if u ∈ infected and random() < β then
                    new_infected ← new_infected ∪ {v}
                    break
                end if
            end for
        end for
        if t > T_trans then
            ρ_rep ← ρ_rep + |infected|/N
        end if
        infected ← (infected \ recoveries) ∪ new_infected
        susceptible ← (susceptible \ new_infected) ∪ recoveries
    end for
    ρ_avg ← ρ_avg + ρ_rep/(T_max − T_trans)
end for
return ρ_avg/N_rep
```

2.2 Microscopic Markov Chain Approach (MMCA)

AIXÒ S'HA DE REVISAR, ESTÀ EXTRET DE LA INFO DEL CHAT. SEGONS EL QUE HI HA IMPLEMENTAT, AQUEST S'AJUSTA MÉS A LES NOSTRES DADES The MMCA [Gómez et al., 2010] provides a deterministic approximation to the SIS dynamics by tracking, for each node i, the probability $\rho_i^{(t)}$ of being infected at time t. The update equation is obtained by writing the Markov transitions exactly:

$$\rho_i^{(t+1)} = \rho_i^{(t)}(1-\mu) + \left(1-\rho_i^{(t)}\right)\left[1 - \prod_{j\in\mathcal{N}(i)}\left(1-\beta\rho_j^{(t)}\right)\right]. \tag{2}$$

The first term on the right accounts for infected nodes that do not recover. The second term accounts for susceptible nodes $(1-\rho_i^{(t)})$ that are infected by at least one neighbour. The key assumption is that the states of different nodes are statistically independent, which allows the joint probability of escaping infection from all neighbors to be factored as a product. Unlike mean-field approaches that linearise this product, MMCA retains the full product form, making it exact on trees and a good approximation on locally tree-like networks.

Starting from $\rho_i^{(0)} = 0.5$ for all i, the iteration is run until

$$\max_i \left|\rho_i^{(t+1)} - \rho_i^{(t)}\right| < \varepsilon, \tag{3}$$

with $\varepsilon = 10^{-10}$, and the global density $\rho = \frac{1}{N}\sum_i \rho_i$ is recorded.

For computational efficiency the product over neighbors is evaluated using the log-sum trick:

$$\prod_{j\in\mathcal{N}(i)} (1-\beta\rho_j) = \exp\left(\sum_j A_{ij}\log(1-\beta\rho_j)\right) = \exp\big(A\cdot\log(1-\beta\rho)\big)_i, \tag{4}$$

reducing the per-iteration cost to a single sparse matrix–vector multiplication.

Linearizing the update equation around $\rho = 0$ shows that the endemic fixed point bifurcates when the spectral radius of βA equals μ, yielding the epidemic threshold

$$\beta_c^{\mathrm{MMCA}} = \frac{\mu}{\lambda_{\max}(A)}. \tag{5}$$

2.3 Quenched Mean Field (QMF)

AIXÒ S'HA DE REVISAR, ESTÀ EXTRET DE LA INFO DEL CHAT. The QMF approach [Wang et al., 2003, Chakrabarti et al., 2008] also retains the full adjacency matrix but replaces the product infection probability with a linear mean-field approximation. Writing the continuous-time rate equation:

$$\frac{d\rho_i}{dt} = -\mu\rho_i + \beta(1-\rho_i)\sum_j A_{ij}\rho_j, \tag{6}$$

the term $\beta\sum_j A_{ij}\rho_j$ acts as the effective force of infection on node i. Setting $d\rho_i/dt = 0$ and defining $\theta_i = \sum_j A_{ij}\rho_j = (A\rho)_i$ yields the self-consistent fixed-point equation:

$$\rho_i = \frac{\beta\theta_i}{\mu+\beta\theta_i}. \tag{7}$$

This is solved by the iteration: initialise $\rho^{(0)} = 0.5$, then alternately compute $\theta = A\rho$ and update $\rho_i \leftarrow \beta\theta_i/(\mu+\beta\theta_i)$ until convergence. Each iteration again requires only a single matrix–vector multiplication.

The epidemic threshold is identical to that of MMCA:

$$\beta_c^{\mathrm{QMF}} = \frac{\mu}{\lambda_{\max}(A)}. \tag{8}$$

Because QMF linearizes the infection probability, it overestimates ρ relative to MMCA for β well above threshold. Both methods share the same threshold β_c, so they agree in the critical region but diverge at higher infection rates.

3 Networks used

All experiments are conducted on four networks of $N = 1000$ nodes generated with fixed random seeds for reproducibility.

Erdös–Rényi (ER). In an ER graph $G(N,p)$ each pair of nodes is connected independently with probability p. Choosing $p = \langle k\rangle/(N-1)$ targets a given average degree $\langle k\rangle$. The degree distribution follows a Poisson distribution centred on $\langle k\rangle$. The spectral radius satisfies $\lambda_{\max} \approx \langle k\rangle$, and the epidemic threshold scales as $\beta_c \approx \mu/\langle k\rangle$ (Equation 8 in [Gómez et al., 2010])Aquesta equació més o manco quadra amb els resultats obtinguts. A les comparacions amb els models teòrics, MMCA és més precís que QMF.

Barabási–Albert (BA). BA graphs are grown by preferential attachment. Each new node attaches m edges to existing nodes with probability proportional to their current degree, producing a power-law degree distribution $P(k) \sim k^{-3}$. β_c is lower compared to ER graphs with the same average degree.

Two values of average degree are studied for each model: $\langle k\rangle \approx 4$ and $\langle k\rangle \approx 6$. The detailed statistics of each network are reported in Table 1.

Two key observations can be derived from Table 1. First, for the same $\langle k\rangle$, BA networks have a much larger $\langle k^2\rangle$ and $\lambda_{\max}$ than ER networks, reflecting the presence of high-degree hubs. This translates directly into a lower β_c: BA networks are considerably more vulnerable to epidemic spreading than their ER counterparts. Second, increasing $\langle k\rangle$ from 4 to 6 increases $\lambda_{\max}$ in both models, again lowering the threshold.

Network	N	E	$\langle k \rangle$	$\lambda_{\max}$	β_c (μ=0.2)	β_c (μ=0.4)
ER $\langle k \rangle = 4$	1000	1988	3.98	5.213	0.0384	0.0767
ER $\langle k \rangle = 6$	1000	2991	5.98	7.178	0.0279	0.0557
BA $\langle k \rangle = 4$	1000	1996	3.99	10.823	0.0185	0.0370
BA $\langle k \rangle = 6$	1000	2991	5.98	14.237	0.0140	0.0281

Table 1: Detailed statistics of the four networks used in the experiments. $\langle k \rangle$ is average degree. $\lambda_{\max}$ is the spectral radius of the adjacency matrix. $\beta_c = \mu/\lambda_{\max}$ is the epidemic threshold.

4 Results and Discussion

5 Code Efficiency

The naive Monte Carlo implementation (Algorithm 1) has two main bottlenecks. First, infected and susceptible nodes are stored as Python lists, making membership queries $O(N)$. Second, the inner loop iterates over all susceptible nodes at every time step, even though most of them have no infected neighbour and therefore zero probability of becoming infected.

Algorithm 2 shows the optimized implementation that addresses both issues. The optimized algorithm was first written in Python and then translated to C++ without any algorithmic changes [1].

Algorithm 2 SIS Monte Carlo simulation (optimized)

Require: Graph $G = (V, E)$, parameters β, μ, N_{rep}, R_0, $T_{\max}$, T_{trans}
Ensure: Stationary infection density ρ

```
Pre-compute 𝒩(v) as a set for all v ∈ V                    ▷ done once before the rep loop
ρ_avg ← 0
for rep = 1 to N_rep do
    infected ← random sample of ⌊R_0 · N⌋ nodes, stored as a hash-set
    ρ_rep ← 0
    for t = 1 to T_max do
        if infected = ∅ then break
        end if
        recoveries ← {v ∈ infected : random() < μ}
        candidates ← (⋃_{v∈infected} 𝒩(v)) \ infected        ▷ only susceptible neighbors of infected nodes
        new_infected ← ∅
        for v ∈ candidates do
            m ← |𝒩(v) ∩ infected|                            ▷ number of infected neighbors
            if random() < 1 − (1 − β)^m then
                new_infected ← new_infected ∪ {v}
            end if
        end for
        if t ≥ T_trans then
            ρ_rep ← ρ_rep + |infected|/N
        end if
        infected ← (infected \ recoveries) ∪ new_infected
    end for
    ρ_avg ← ρ_avg + ρ_rep/(T_max − T_trans)
end for
return ρ_avg/N_rep
```

The optimized version introduces four key changes with respect to the naive one:

1. **Hash-set storage.** The infected set is maintained as a hash-set (Python `set` / C++ `unordered_set`), reducing membership checks from $O(N)$ to $O(1)$.

[1] Given that this was not the main scope of the project, the translation to C++ was carried out with the use of LLMs.

2. **Candidate filtering.** Instead of iterating over all susceptible nodes, only the union of the neighbourhoods of currently infected nodes is considered (line 9). When the epidemic is sparse, this set is much smaller than N, providing a large practical speedup.

3. **Compound infection probability.** Rather than drawing a separate Bernoulli trial for each infected neighbour, the probability of being infected by at least one of m infected neighbors is computed in closed form as $1 - (1 - \beta)^m$ (line 12), reducing the number of random number draws.

4. **List comprehensions (Python only).** The recoveries and candidate sets are built with Python set/list comprehensions instead of explicit `for` loops. Comprehensions execute in CPython's internal C loop, which is faster than interpreted Python loops.

5.1 Timing comparison

Table 2 reports the total wall-clock time to run all 248 simulation points (4 networks $\times$ 2 values of μ $\times$ 31 values of β) with $N_{\text{rep}} = 100$ on the same hardware [2]. Three implementations are compared: the naive Python baseline, the optimized Python version applying the three changes described above, and the C++ port of that same optimized algorithm (identical logic, no further algorithmic changes).

Implementation	Total time	Speedup vs. naive	Speedup vs. efficient Py
Naive Python	$\approx$53544 s ($\sim$14.9 h)	1$\times$	—
Efficient Python	$\approx$4606 s ($\sim$1.3 h)	11.6$\times$	1$\times$
Efficient C++	$\approx$1162 s ($\sim$19 min)	46.1$\times$	$\sim$4$\times$

Table 2: Total wall-clock time for the full simulation campaign (248 points, $N_{\text{rep}} = 100$ each). Speedup is relative to the naive Python baseline.

The optimized Python implementation is roughly 11.6$\times$ faster than the naive version. The C++ implementation applies the same algorithmic optimisations as the efficient Python version, but takes advantage from compiled execution, native data structures, and the absence of interpreter overhead. This yields an additional $\sim 4\times$ speedup over efficient Python, bringing the total reduction to $\sim$46$\times$ relative to the naive baseline. The C++ run completes all the experiments in under 20 minutes, compared to nearly 15 hours for the naive implementation.

5.2 Implementation correctness

Despite the algorithmic differences described above, the three implementations are equivalent. The slight discrepancies visible in Figure 1 are purely due to Monte Carlo sampling variance. Each implementation consumes random numbers at a different rate per time step (the naive version draws up to one number per infected neighbour per susceptible node and the optimized version draws exactly one per candidate), so despite sharing the same initial seed the two random streams diverge immediately, producing different stochastic trajectories. The discrepancy is most visible at large β, where the endemic state fluctuates more across realizations, amplifying the error.

6 Conclusion

[2] All simulations were run on a Linux machine, running Ubuntu with an AMD Ryzen 9 7900X 12-core processor

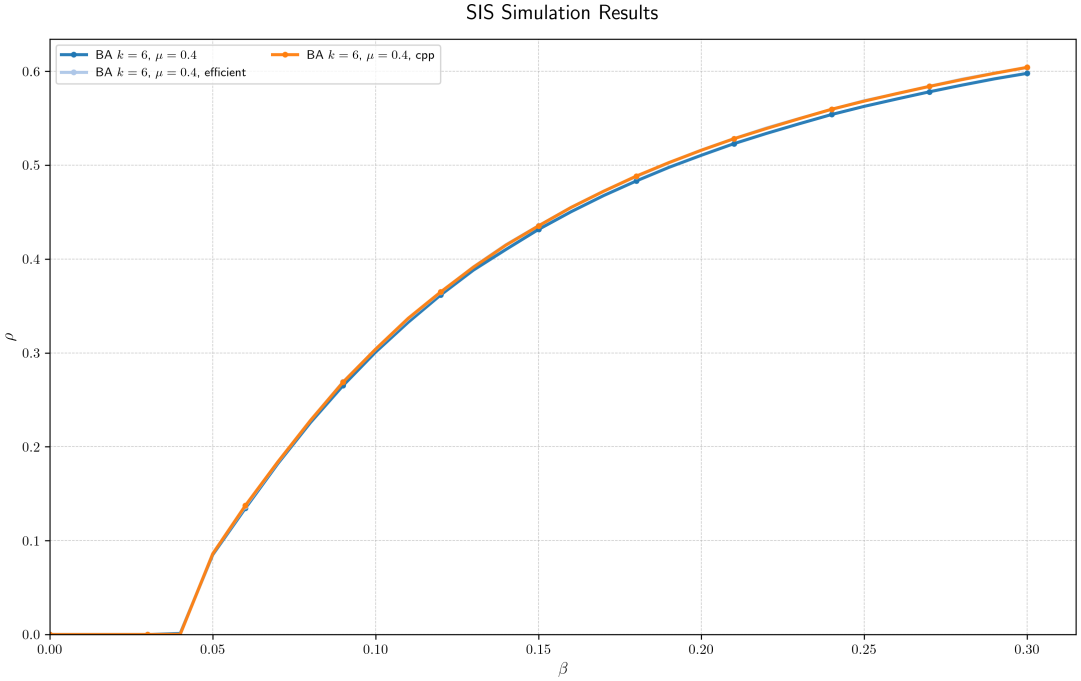

Figure 1: Stationary infection density ρ as a function of β for the BA network with $\langle k \rangle = 6$ and $\mu = 0.4$, estimated by the three implementations. The curves overlap almost perfectly. Differences are due to Monte Carlo sampling variance from finite $N_{\text{rep}} = 100$.

References

[Chakrabarti et al., 2008] Chakrabarti, D., Wang, Y., Wang, C., Leskovec, J., and Faloutsos, C. (2008). Epidemic thresholds in real networks. *ACM Transactions on Information and System Security*, 10(4):1–26.

[Gómez et al., 2010] Gómez, S., Arenas, A., Borge-Holthoefer, J., Meloni, S., and Moreno, Y. (2010). Discrete-time Markov chain approach to contact-based disease spreading in complex networks. *Europhysics Letters*, 89(3):38009.

[Wang et al., 2003] Wang, Y., Chakrabarti, D., Wang, C., and Faloutsos, C. (2003). Epidemic spreading in real networks: An eigenvalue viewpoint. pages 25–34.